

JACK THOMAS

Redondo Beach, CA 90278 | Mobile: (424) 241-0782 | Email: jack@jack-thomas.com |
<https://www.linkedin.com/in/jack-thomas-22a3a1117/> | <https://github.com/jackthomas00>

Software Developer

New-grad software engineer with experience building and deploying production web applications using Django, PostgreSQL, React, and Docker.

EDUCATION

Bachelor of Science Degree, Computer Science - California State University Fullerton, May 2026

Relevant Coursework: Data Structures & Algorithms, Operating Systems, Database Systems, Computer Networks, Software Engineering, Machine Learning, Computer Architecture, Discrete Mathematics

TECHNICAL SKILLS

Languages: Python, C++, JavaScript, TypeScript, SQL

Frameworks/Tools: React, Node.js, Git, Docker, Django, REST APIs

Databases/Cloud: PostgreSQL, AWS (EC2, S3, Lambda)

Concepts: Data Structures & Algorithms, OOP, Agile/Scrum, CI/CD, Unit Testing

EXPERIENCE

FULLSTACK SOFTWARE DEVELOPER (internship)

2024 – Present

TradeCRE, Redondo Beach, CA

- Built a secure Django/PostgreSQL backend for multi-party real estate transactions, implementing fine-grained RBAC across 6 resource types for brokers, buyers, and legal teams.
- Developed comprehensive automated Django test suite covering authentication, permissions, serializers, and encrypted transaction workflows to maintain reliability across weekly production deployments.
- Achieved sub-100ms median response times across 15+ core API endpoints in production; heaviest data endpoints (deals, files) consistently served under 110ms.
- Containerized the full production stack (Django, PostgreSQL, Nginx) with Docker and shipped weekly production updates; collaborated with a domain expert (real estate) to translate business requirements into product features, iterating on workflows and UX to make the platform usable for brokers and buyers.

SOFTWARE DEVELOPER (personal project)

2025 – 2025

OKC, Redondo Beach, CA

- Built a domain-agnostic NLP pipeline that compiles unstructured text into a queryable multi-layer knowledge graph with entities, claims, typed relations, and provenance.
- Ingested and processed 5,000+ documents with fault-tolerant scraping (429/503 back-off), deduplication via content hashing, and restartable stage-skipping to ensure idempotent, large-scale ingestion without full reprocessing.
- Shipped a modular monorepo (okc_core, okc_pipeline, okc_api, okc_ui, okc_cli) with a FastAPI REST layer, Postgres + pgvector backend, and a React explorer UI — fully orchestrated via Docker Compose.